

Generic Programming for Multidimensional Arrays

Alfredo A. Correa

A dark green silhouette of a mountain range spans the bottom half of the slide, serving as a background for the author's name.

Material for MULTI library

Alfredo A. Correa

June 1, 2026

- 1 Introduction & Motivation
- 2 Generic Programming Foundations
- 3 Array Model
- 4 Lazy Arrays & Restrictions
- 5 Algorithms & Performance
- 6 Adaptors & Ecosystem

- Physicist (Condensed Matter Physics)
- Group Leader at the *Quantum Simulations Group* at Lawrence Livermore Lab (California)
- Simulating Quantum Mechanics using Classical Computers, HPC, and GPUs
- Research oriented
- INQ Electronic structure code for ground state and time-dependent (C++/MPI/CUDA)
“A Modern GPU-Accelerated Computational Framework for (Time-Dependent) Density Functional Theory”, Andrade et al. J.Chem.Theory Comput. (2021)
- QMCPACK Quantum Monte Carlo for accurate energies of molecules

- Condensed Matter Physics (materials simulations)
- Community is historically FORTRAN-based
- Always in search of the perfect language for physical simulations (expressiveness, correctness, high performance)
Mathematica, C, Smalltalk, Matlab
- Many false starts with C++: object-oriented design, pointless template metaprogramming, virtual polymorphism, multiple dispatching, DSLs...
- ...until I found Stepanov's works, understood the STL, all clicked... start of a journey

Can generic programming compete with array languages? (Fortran, APL, Matlab, NumPy)

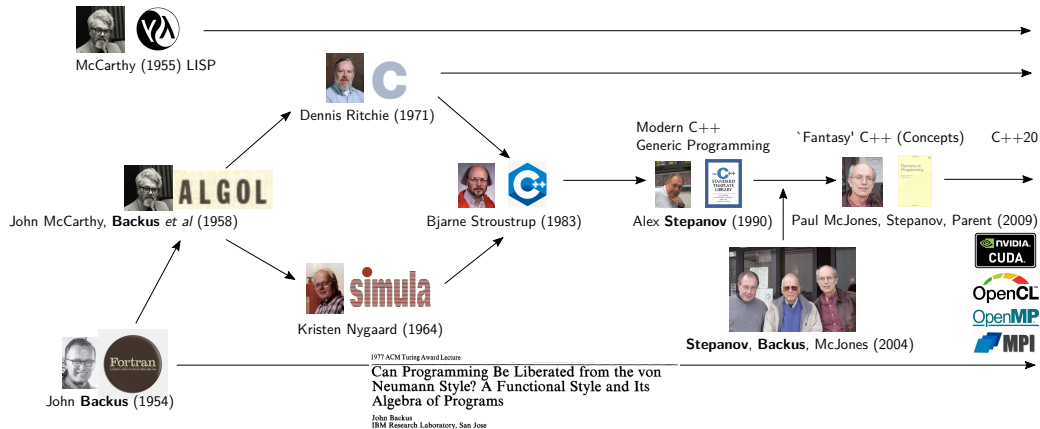
- LLNL Representative at the C++ Standards Committee (secondary/alternate)
- First LEWGI draft P3879 on `std::overload` (with Branden Ganetsky)
- libraries: Boost.Multi (this work), B.MPI3, B.Covariant (func. prog. with variant types)

Family tree of languages at Computer History Museum, San José, **history1**



Don't worry, we are still programming in Fortran, sort of.

history2



"John [Backus] was very impressed... he always knew that... he needed to figure out how to work mutation into functional programming and... generic programming is functional programming with a well-defined way to handle mutation." (Sean Parent, 2018, 'Greatest Common Measure', youtu.be/FtZEU9zv9eM)

Operation	Fortran 90	C++ Multi
Declaration (3×3)	<code>real(8):: A(3,3)</code>	<code>multi::array<double,2> A({3,3});</code>
Allocatable	<code>real(8), allocatable :: A(:, :)</code>	<code>multi::array<double,2> A;</code>
Allocation	<code>allocate(A(10,5))</code>	<code>A.reextent({10,5});</code>
Element access	<code>A(i,j)</code>	<code>A[i][j]</code> or <code>A(i,j)</code> (zero-index)
First row	<code>A(1,:)</code>	<code>A[0]</code> or <code>A(0,_)</code>
First column	<code>A(:,1)</code>	<code>A(_,0)</code>
Subblock	<code>A(1:2, 1:2)</code>	<code>A({0,2},{0,2})</code>
Deallocation	<code>deallocate(A)</code>	<code>A.clear()</code>

Key differences

Fortran indices start at **1**, Multi at **0**; ranges `1:2` are closed, `{0,2}` is half-open.

Operation	NumPy	C++ Multi
Uninitialized (3×3)	<code>A = np.empty((3,3))</code>	<code>multi::array<double,2> A({3,3});</code>
Zero-filled (3×3)	<code>A = np.zeros((3,3))</code>	<code>multi::array<double,2> A({3,3}, 0.0);</code>
Element access	<code>A[i,j]</code>	<code>A[i][j]</code> or <code>A(i,j)</code> (or in C++23 <code>A[i,j]</code>)
First row	<code>A[0]</code> or <code>A[0,:]</code>	<code>A[0]</code>
First column	<code>A[:,0]</code>	<code>A(_,0)</code>
Subblock	<code>A[0:2, 0:2]</code>	<code>A({0,2},{0,2})</code>
Strided slice	<code>A[:,::2, :]</code>	<code>A.strided(2)</code>
Shape / extent	<code>A.shape</code>	<code>A.extensions()</code>
View (zero-copy)	<code>B = A[0:2]</code> (view)	<code>auto&& B = A({0,2});</code> (subarray)
Copy	<code>B = A[0:2].copy()</code>	<code>auto B =+ A({0,2});</code>
GPU array	<code>cupy.array(...)</code>	<code>multi::thrust::array<double,2></code> (type alias)

Key differences

NumPy slices use **closed-open** `[0:2]` ranges — same as Multi `{0,2}`. Multi subarrays are **always views**; a copy requires an explicit syntax. Multi's type encodes rank and element type at compile time ⇒ zero overhead, GPU portability.

What kind of programs are not mathematical? Programs that are insecure, programs that crash, programs that cannot be easily modified are examples of non-mathematical software. There is, of course, a solid economic reason for producing such programs: they assure world domination on a large scale and job security on a small scale. – Alex Stepanov, Notes on the foundations of programming, 2005

[Guys!] We should use formal methods even when we write XML stuff. – Alex Stepanov (2003), youtu.be/fanm5y00joc

Scientific computing is array computing

- Discretized PDEs: 2D/3D grids
- Quantum chemistry: matrices, tensors
- Signal processing: time $\times$ frequency
- Machine learning: batches $\times$ features $\times$ channels

Performance lives in arrays

- Contiguous/Regular memory $\Rightarrow$ cache efficiency
- SIMD vectorization, GPU parallelism
- BLAS/LAPACK/FFTW operate on array layouts

Arrays are a natural abstraction

- Most algorithms iterate over index ranges
- Lingua franca: Fortran, APL, Matlab, NumPy built around them
- C++ STL: ranges of elements — but only 1D

The gap (A blessing in disguise)

C++ has no standard multidimensional array with value semantics, subarrays, and algorithm support.

Motivating example: Electron density ρ reacts to swift atomic motion

motivation2

real numbers
(electron density)

complex numbers

elementwise

$$\rho(\mathbf{r}, t) = \sum_i^{N_{\text{electrons}}} |\psi_i(\mathbf{r}, t)|^2$$

4D reduction 5D

PDE for ψ

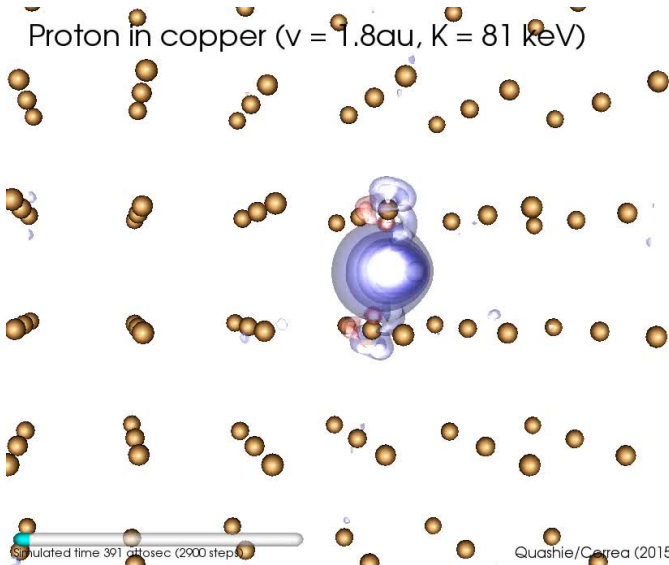
FFT

linear alg

GPU

MPI comm

Proton in copper ($v = 1.8\text{au}$, $K = 81\text{ keV}$)



Quashie/Correa (2015)

What are multidimensional Arrays

intro1

A

1D

0	1	2
a	b	c

B

2D

	0	1	2
0	i	j	k
1	l	m	n

C

3D

	0	1	2	
0	o	p	q	2
1	r	s	t	w
1	x	y	z	

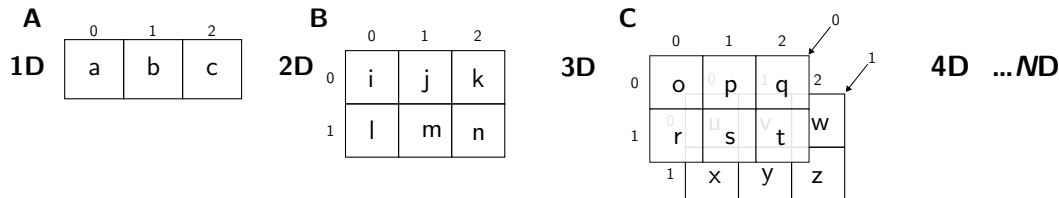
4D ...ND

```
assert( A[1] == 'b' );           // A(1)
assert( B[1][2] == 'n' );       // B(1, 2)
assert( C[1][0][2] == 'w' );    // C(1, 0, 2)
```

Visualization is only a convention

Implies a symmetry in the indices.

Multidimensional Arrays



```
assert( A != B[0] );  
assert( C(0, {0, 2}, {0, 2}) != B );
```

Expected behavior across dimensions

There are relations across dimensions

note: most examples in this talk are 2D for simplicity, imagine the ND generalization

Keynote: “Making C++ Standard Parallelism Multidimensional”

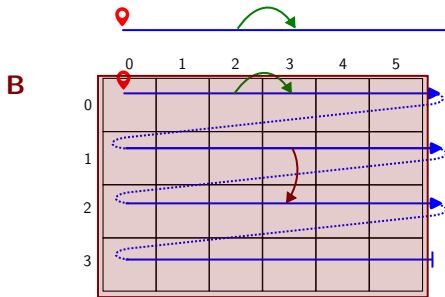
- Do not miss this keynote by Mark Hoemmen (NVIDIA)
- ‘Typed linear algebra’ by François Carouge
- PARROT library by Conor Hoekstra (NVIDIA) github.com/NVlabs/parrot
- CUDA Tile soon to be available for C++
- BOOST.MULTI (this work) fully accepted? 🙌

2026

Is 2026 **the year of C++ as an array language?** 😊

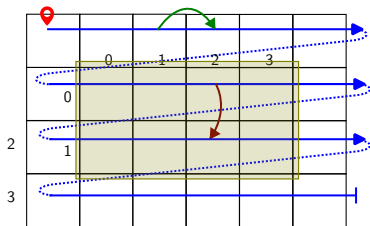
Strided arrays as a data structure, subarray subblock

strided1



```
{  
  .base =   
  .stride1 = 6  
  .stride2 = 1  
  .nelem1 = 24  
  .nelem2 = 6  
}
```

$B(\{1, 3\}, \{1, 5\})$

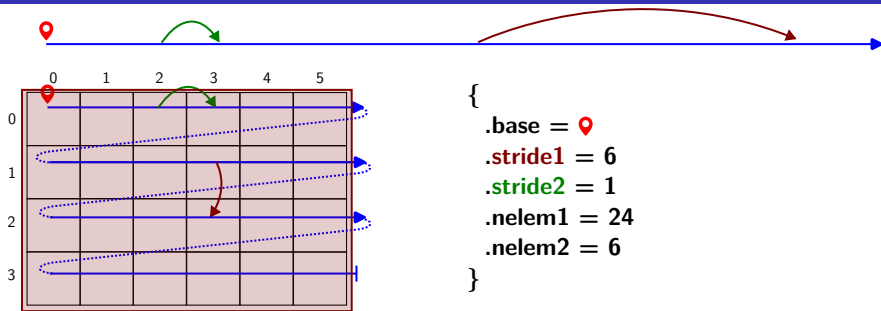


```
{  
  .base =  + 7  
  .stride1 = 6  
  .stride2 = 1  
  .nelem1 = 12  
  .nelem2 = 4  
}
```

Strided arrays as a data structure, row subarray

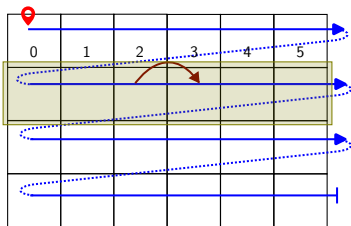
strided2

B



```
{  
  .base =   
  .stride1 = 6  
  .stride2 = 1  
  .nelem1 = 24  
  .nelem2 = 6  
}
```

$B[1] = B(1, _)$

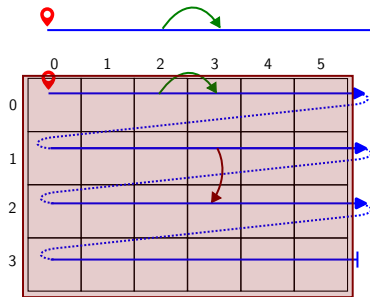


```
{  
  .base =  + 6  
  .stride1 = 1  
  
  .nelem1 = 6  
}
```

Strided arrays as a data structure, column subarray

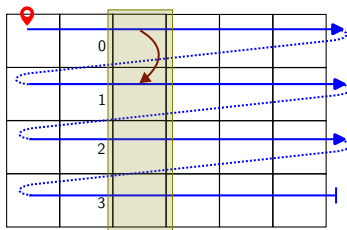
strided3

B



```
{  
  .base =   
  .stride1 = 6  
  .stride2 = 1  
  .nelem1 = 24  
  .nelem2 = 6  
}
```

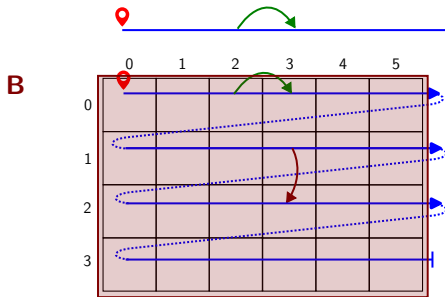
$$B^T[2] = B(_, 2)$$



```
{  
  .base =  + 6  
  .stride1 = 6  
  
  .nelem1 = 24  
}
```

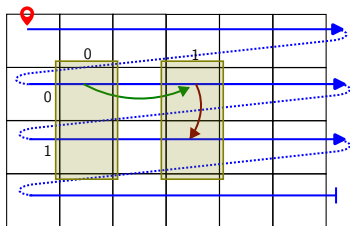
Strided arrays as a data structure, strided subarray

strided4



```
{  
  .base =   
  .stride1 = 6  
  .stride2 = 1  
  .nelem1 = 24  
  .nelem2 = 6  
}
```

B({1, 3}, {1, 5, 2})



```
{  
  .base =  + 7  
  .stride1 = 6  
  .stride2 = 2  
  .nelem1 = 6  
  .nelem2 = 4  
}
```

element $i, j \rightarrow \text{base} + \text{stride1} * i + \text{stride2} * j$

Pros:

- Maintains symmetry as much as possible between dimensions
- Memory of arrays and subarrays is compact, regular, and often contiguous
- Compatible with legacy libraries, FFTW, BLAS, LAPACK, cuFFT, cuBLAS, MPI

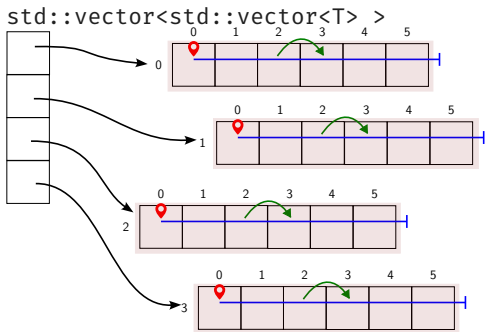
Cons:

- Not available in the language; needs library solutions
- No support from STL algorithms
- Not easy to grow

Other data structures (layouts) also maintain symmetry

Tiled, Zig-zag, Hilbert curve filling

Multidimensionality via nested containers (the wrong way)



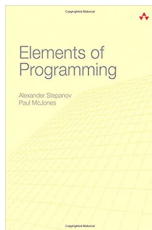
Pros (mostly traps)

- `B[i][j]` works out of the box
- Iterator access available (but only if you access things the right way)
- Easy to grow (in the first dimension)

Cons:

- Distinct logic for different dimensions
- Memory all over the place
- Has to maintain invariant (rectangular size)
- Not compatible with legacy libraries
- Gets worse with higher dimensionality

Semiregular types (Stepanov Regularity, EoP)



```
struct SR {  
    SR(); // default constructible  
    SR(SR const& other); // copy constructible  
    ~SR(); // destructible  
    auto operator=(SR const& r)  $\rightarrow$  R&; // assignment  
};
```

... and also the semantics should be as expected

Available algorithms: swap (std::swap)

Regular = Semiregular + has equality

```
struct R { ...  
    auto operator==(R const& other) const → bool;  
    auto operator!=(R const& other) const → bool;    // neg  
};
```

in C++20

```
struct R { ...  
    auto operator==(R const& other) const → bool;    // = default;  
    // ^^ synthesizes !=  
};
```

Some types might not have equality in practice because equality could be prohibitively expensive or the algorithm for equality is not known.

Available algorithms: linear-search (`std::find`)

PartialOrder = Regular + has order

```
struct PO { ...  
    bool operator< (PO const& other) const;  
    bool operator> (PO const& other) const;  
    bool operator>=(PO const& other) const;  
    bool operator<=(PO const& other) const;  
};
```

in C++20

```
struct PO { ...  
    auto operator== (P const& other) const = default;  
    auto operator< (P const& other) const = default;  
    auto operator<==>(P const& other) const = default;  
    // ^^ synthesizes <=, >=, >,  
};
```

Available algorithms: sort (std::sort), binary-search (std::lower_bound)

TotalOrder = PartialOrder + has trichotomy

trichotomy: Not $a < b$ and not $a > b \Rightarrow a == b$

```
struct TO { ...  
    bool operator< (TO const& other) const;  
    bool operator> (TO const& other) const;  
    bool operator>=(TO const& other) const;  
    bool operator<=(TO const& other) const;  
};
```

in C++20

```
struct TO { ...  
    auto operator< (TO const& other) const = default;  
    auto operator<=(TO const& other) const = default;  
    // ^^ synthesizes ==, !=, >, <=, >=  
};
```

Available algorithms: same as before, but they can be more efficient.

Claim: (Semi)Regular types are amenable to equational reasoning.

Modern C++ is based around them

Litmus test for your specific type `Type`

Quick test for your types, does this compile? Does it do the expected thing?

```
std::vector<Type> v = {r1, r2, r3};  
std::sort( v.begin(), v.end() );  
assert( std::is_sorted(v.begin(), v.end() ) );
```

A number of classes in libraries (or frameworks) do **not** pass the litmus test.

```
coocoolib_array a; coocoolib_array b;  
a = b; // look how smart (?), a now "points" to b  
a.copy(b); // inflation of syntax  
a.deep_copy(); a.really_copy(b); a.NoJoke_copy(b);
```

Values = Regular + platonic / persistent

Values are Regular types that are also amenable to mathematical, equational, logical reasoning, e.g. arithmetic. Mathematics and numerical mathematics are based around them.

Values = Regular + saving/loading to a medium (serialization)

```
struct Value { ...  
    void save(Archive& ar) const;  
    void load(Archive& ar);  
};
```

```
void print(Value const&); void read(Value &);
```

```
auto operator<<(std::ostream& out, Value const& self) {...}  
auto operator>>(std::istream& out, Value & self) {...}
```

Drawback

There is no standard way to serialize in C++. It is up to us to adopt a convention, protocol(s) or a standard.

MULTI was proposed to Boost (2026) and conditionally accepted

boost1

gitlab.com/correea/boost-multi

Project: boost-multi

Project information

Multidimensional arrays for C++. (Conditionally accepted Boost library) GitHub mirror: <https://github.com/correea/boost-multi>

C++ multidimensi... array + 7 more

coverage 99.16% pipeline passed Try me online! CE Latest Release v0.86.0 License boost 1.0 0 active issues 15.5

security 100% maintainability A code smells 2.6k coverage 97.9% CMake passing Coplanning #boost-multi CI passing

documentation HTML as single header Presentation PDF

14,625 Commits 235 Branches 11 Tags 34.5 GiB Project Storage 9 Releases

github.com/correea/boost-multi

correea / boost-multi

Code Issues 26 Pull requests 9 Agents Discussions Actions Projects Wiki Security and quality 7 Insights Settings

boost-multi Public

Unpin Unwatch 4 Fork 3 Starred 18

remove-some-parent-clang-tidy-gb had recent pushes 20 minutes ago Compare & pull request

develop 610 Branches 11 Tags Go to file Code

correea Merge branch 'remove-some-parent-clang-tidy-gb' into... 94cb5a0 · 7 hours ago 14,625 Commits

File	Commit	Time
.github	Add dependabot workflow	2 months ago
.gitlab/merge_request_templates	Edit default.md	7 months ago

correea.github.io/boost-multi

Boost.Multi

Introduction

Installation

Getting Started

Advanced Usage

Motivation

Manipulation of Non-Managed Memory

Initialization

Copy, Assignment, and Aliasing

Change Sizes

Iteration

Indexing

Element Access and Partial Access

Slices and Strides

Conversions

Uninitialized vs. Initialized Elements

Compile-time Evaluation

Restrictions

Elementwise Expressions

Interoperability

STL Algorithms

Range-v3

Serialization

Boost.Interprocess

CUDA

OpenMP via Thrust

CUDA C++

SYCL

(fmt)

Python (cppyy)

Legacy C libs

Technical points (design rationale)

Indexing

Iteration

Minimum C++ version

Introduction

Multi is a modern C++ library that provides manipulation and access of data in multidimensional arrays for both CPU and GPU memory.

Multidimensional array data structures are essential in various fields of computing, including data analysis, image processing, and scientific simulations. When combined with GPUs, they also play a significant role in Artificial Intelligence and Machine Learning.

In many simulations, space is divided into a grid, and values are stored at each grid cell. For example, arrays can represent fields of physical values, such as temperature variations within a block of material, or the velocity field of a fluid. The state of a game can be represented by placing pieces in a board made of squares. Arrays can also be used to numerical matrices, vectors, and more generally, tensors, to perform linear algebra operations. They can represent other abstract data structures, like networks, graph connections, or store data in multiple columns. Arrays are amenable to bulk processing in computing, and also to slicing, and reductions.

Here are 3 examples of arrays of different dimensionalities:

1D	2D	3D																			
<table border="1"><tr><td>a</td><td>b</td><td>c</td></tr></table>	a	b	c	<table border="1"><tr><td>0</td><td>f</td><td>g</td><td>h</td></tr><tr><td>1</td><td>i</td><td>j</td><td>k</td></tr></table>	0	f	g	h	1	i	j	k	<table border="1"><tr><td>0</td><td>f</td><td>g</td><td>h</td></tr><tr><td>1</td><td>i</td><td>j</td><td>k</td></tr></table>	0	f	g	h	1	i	j	k
a	b	c																			
0	f	g	h																		
1	i	j	k																		
0	f	g	h																		
1	i	j	k																		

Note that the depicted 3D case has only two planes, the first index (e.g. 1) selects a plane (e.g. bottom).

These depicted 1D, 2D, and 3D arrays containing letters and can be represented, within this library, as C++ objects declared as:

```
#include <boost/multi.hpp>
multi::array<char, 1> A1D = {'a', 'b', 'c'};
```

Thanks Matt Borland @ CPPAlliance!

Why **not** proposed for Standard (or Beaman)?

- Can't promise (ABI) compatibility
- Controversial idioms (some hacks) 🌶️
- Too restrictive for compatibility with exotic 3rd parties (Thrust, BLAS, CUDA, cuLibs)
- Enough overlap with `std::span`, `std::mdspan` and `std::mdarray`
- Long time horizon (I am stuck in C++17/20 for compatibility with GPUs)

Why proposed to Boost?

- Ideal balance between risk (innovation) and requirements (quality, guidelines)
- Peer reviewed
- Still good distribution model and visibility
- No need for compiler-magic
- Moderate time horizon
- Can eventually benefit from low-level Boost tools (Boost.Hana, Boost.Assert, Boost.Align, Boost.STLInterfaces)
- Peer interaction with Boost.Serialization, Boost.JSON, Boost.QVM, Boost.YAP

- INQ: engine for electronic structure calculations and dynamics, LLNL
- QMCPACK: Quantum Monte Carlo (QMC) simulation code, Oak Ridge Natl Lab
- CoQuí: Correlated Quantum Systems (post-DFT: GW, RPA, THC-AFQMC) Flatiron Institute, NY
- Dalotia: data loader library for tensors in AI, RIKEN (RCCS), Japan

Abinitio C++ Codes INQ (DFT/TDDFT) & QMCPACK (QMC/AFQMC) uses2

The left screenshot shows the GitHub repository for 'inq' by NPNEQ. It displays a commit history table with columns for Name, Last commit, and Last update.

Name	Last commit	Last update
ci	directly test ci and not libxc	1 year ago
cmake	removed manual file	8 months ago
examples	Merged input: basis into syste...	1 month ago
externalLibs	Updated multi to the last versio...	1 month ago
scripts	Add spglib to inc++	5 months ago
share/unit_tests_data	Removed some redundant alu...	5 months ago
src	Use lowest instead of min, as s...	1 day ago
tests	Created the acces function phi(...	1 month ago
.configure-custom.sh	Add an option to specify the bo...	10 months ago
.gitignore	don't show built.* with git	4 months ago
.gitlab-ci.yml	Increased the timeout time for ...	1 month ago

The right screenshot shows the GitHub repository for 'qmpack' by QMCPACK. It displays a list of files and a recent pull request.

Files:

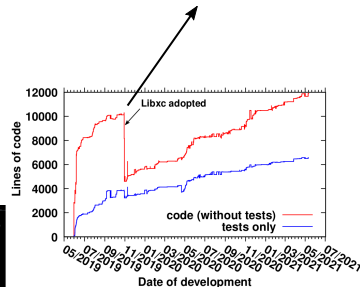
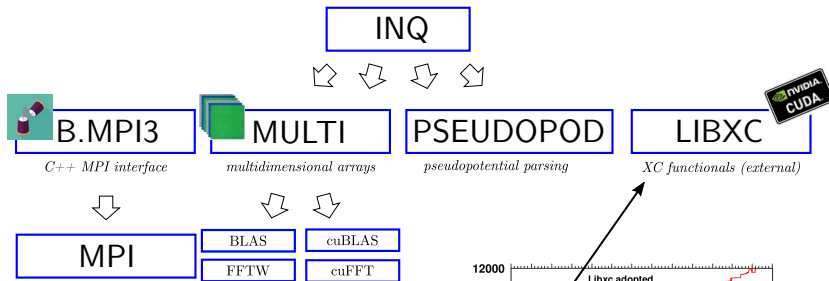
- .github
- CMake
- build
- config
- docs
- doxygen
- examples
- external_c...
- labs
- nexus
- schema
- src

Recent pull request:

- ye-luo Merge pull requ... 26 minutes ago 21,789

Observation

C++ has become the de facto standard for HPC. We better create a good ecosystem.



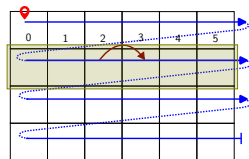
Individual components can be shared and reused by other projects

```
#include <boost/multi/array.hpp>
namespace multi = boost::multi;
// from extents
multi::array<int, 2> A({3, 4}); // a 3 x 4 array

// from a (rectangular) initializer list
multi::array<int, 2> B = {{1, 2, 3}, {4, 5, 6}};

// from existing memory (no allocation, no ownership)
int data[12] = {};
multi::array_ref<int, 2> C({3, 4}, data);

// element access
A(1, 2) = 5; // equivalent
A[1, 2] = 5; // equivalent C++23
A[1][2] = 5; // intermediate object?
```



```
{
    .base = 0 + 6
    .stride1 = 1

    .nelem1 = 6
}
```

```
multi::array<int, 2> A({4, 6});
```

```
??? row = A[1]; // same as A(1, _) or *(A.begin() + 1)
```

```
// choose one:
```

auto	row = ...	// misleading, valid but not recommended
multi::subarray<double, 1>	row = ...	// too verbose, but correct
multi::array<double, 1>	row = ...	// correct, but makes a copy
auto&&	row = ...	// self-documenting

The problem with auto: “Implicit Evaluation of ‘auto’ Variables.”

P0672R0 (2017) by Joël Falcou, Herb Sutter et al. (std::vector<bool>, proxy objects)

- Use the provided 'auto-plus' operator
- Rely on lifetime extensions 🌶️ to document naming of reference-subarrays

```
multi::array<double, 2> A({3, 4});
```

```
auto      row = + A[1]; // multi::array<double, 1>      row = A[1];
auto&&    row =  A[1]; // multi::subarray<double, 1>    row = A[1];
auto const& row =  A[1]; // multi::const_subarray<double, 1> row = A[1];
```

Prefer forward (universal) references:

```
template<class Array2D>
    // constrains, e.g. dimensionality == 2
auto f(Array2D&& arr) { ... }
```

Want to avoid bloat? (or work across compilation boundaries?)

```
auto g(multi::subarray<int, 2> arr) { ... }
```

Indexing

- Index
`A[i]` removes (binds) first dimension
- Slice
`A.sliced(first, last)` range subarrays (equal dimension) half-open interval `[first, last)`

Fundamental

- Rotate (indices)
`A.rotated()[i][j][k] == A[k][i][j]`
- Unrotate (indices)
`A.unrotated()[i][j][k] == A[j][k][i]`
- Transpose (first two indices)
`A.transposed()[i][j][k] == A[j][i][k]`
- Strided `A.strided(n)[i] == A[n*i]`

- Indexing (with parentheses) $A(i) \rightarrow A[i]$
- Sliced (terse) $A(\{first, last\}) \rightarrow A.sliced(first, last)$
- Generalized slice:
 $A(\{first, last, every\}) \rightarrow A.sliced(first, last).strided(every)$
- Row
 $A(i, _) \rightarrow A[i](_) \rightarrow A[i]$
- Column
 $A(_, i) \rightarrow A.rotated()[i]$
- Partial column
 $A(\{first, last\}, i) \rightarrow A.rotated()[i](\{first, last\})$

Too many temporaries objects?

Modern compilers can elide these temporary objects.

Since C++23 you can use multidimensional square brackets $A[i, j, k]$

Aliasing and how to break it

```
multi::array<int, 2> A({3, 3});  
multi::array<int, 2> B({3, 3});  
multi::array<int, 2> C;  
  
B = A.transposed(); // ok allocates?  
C = A.transposed(); // ok allocates?  
  
A = A.transposed(); // ok? aliasing!  
  
A = + A.transposed(); // ok allocates?
```

Dual purpose of operator+

- Help auto
- Materialize [sub]array-references

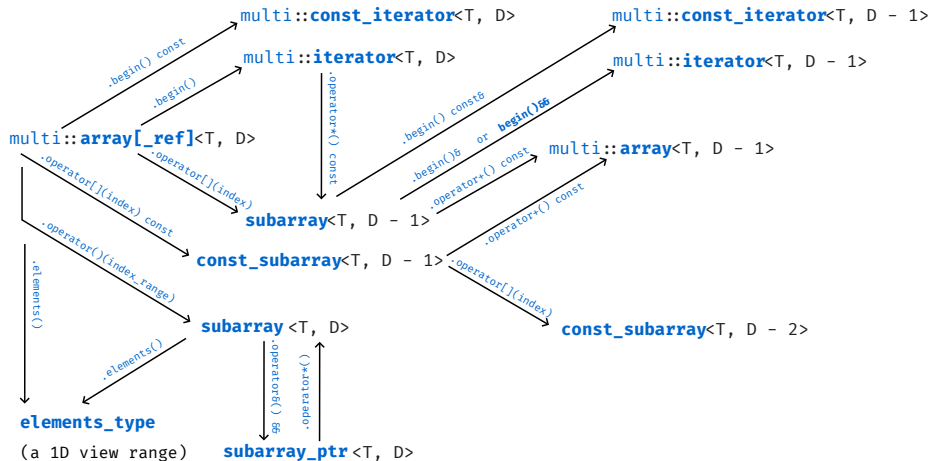
Type hierarchy: ownership and mutability

Type	Owns	Copyable	Move	Swap	Mutable elems?	Resizes
<code>multi::const_subarray<T, D, P></code>	No	No 🌶️	No	No	No	No
↑						
<code>multi::subarray<T, D, P></code>	No	No 🌶️	No	O(N)	Yes	No
↑						
<code>multi::array_ref<T, D, P></code>	No	No 🌶️	No	O(N)	Yes	No
↑						
<code>multi::dynamic_array<T, D, A></code>	Yes	Yes	No	O(N)	Yes	No
↑						
<code>multi::array<T, D, A></code>	Yes	Yes	Yes	Yes	Yes	Yes

Inheritance for composition (usually a bad idea)

I made it public inheritance 🌶️ because I found it useful. It is opt-in.

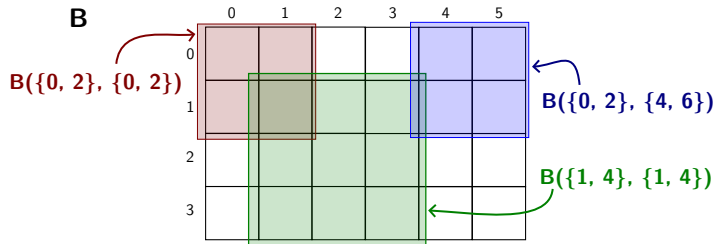
Relations between types, recursion across dimensions (simplified)



It looks complicated

Intuitive for the most part in practice. Perhaps arrays deserve this level of complexity.

Wait! `multi::subarray`'s are not copyable? How to "store" subarrays?



```
std::vector sas = {B({0, 2}, {0, 2}), B({1, 4}, {1, 4}), B({0, 2}, {4, 6})};  
// error! subarray<int, 2> is not copyable
```

```
std::vector saps = {&B({0, 2}, {0, 2}), &B({1, 4}, {1, 4}), &B({0, 2}, {4, 6})};  
// std::vector<multi::subarray_ptr<T, 2>> sarrps = ...  
for(auto sap : saps) {  
    std::ranges::fill(sap->elements(), 0);  
}
```

“Copy or copy not; there is no shallow. – Master Yoda” – Tony Van Eerd

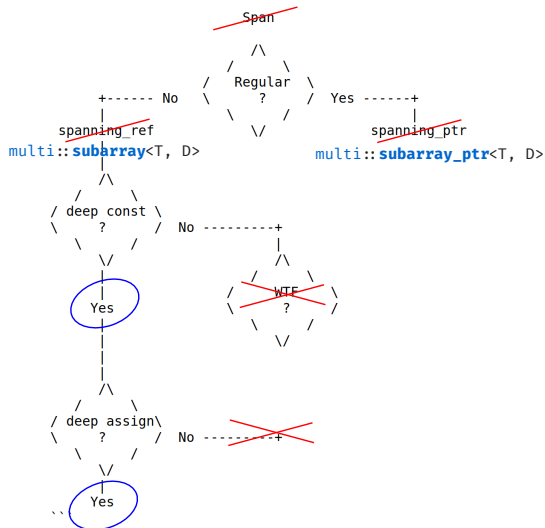
Should Span be Regular?

Document number: P1085R0

Date: 2018-05-04

Audience: EWG

Reply-to: Tony Van Eerd. regular at forecode.com



multi::dynamic_array: pinned owning array, multi::inplace_array

multi::dynamic_array<T, D, A> owns memory, is copyable, but **not movable**:

```
multi::dynamic_array<double, 2> D({3, 4}); // allocates, fixed size
multi::dynamic_array<double, 2> E = D;      // ok: deep copy
multi::dynamic_array<double, 2> F = std::move(D); // ok, but O(N)
```

- Size is fixed at construction (no resize, no reallocation)
- Iterators and pointers are **never invalidated** after construction
- Iterators, subarrays, and pointers never invalidated unless the object is destroyed

Inspired by std::dynarray (proposed for C++14, N3467, Stroustrup et al.)

std::dynarray was proposed as a fixed-size heap array but did not make it into the standard. multi::dynamic_array generalizes the idea to N dimensions.

```
template<class T, dimensionality_type D, std::size_t Capacity = 32>
using inplace_array = multi::dynamic_array<T, D, multi::detail::static_allocator<T,
    Capacity>>>;
```

Looks complicated, does it work? Test 1/2

Regularity of `multi::array<T, D>`:

The *litmus* test for Regular types:

```
std::vector<multi::array<int, 2>> V = { a1, a2, a3 };  
std::sort( V.begin(), V.end() );  
std::is_sorted( V.begin(), V.end() );
```

godbolt.org/z/df634c63G

Passes!

Arrays have total order (if elements have total order), and behave like Regular

The test is not about sorting... (not often used for arrays)

... it is about showing that all the intermediate components work.

Looks complicated, does it work? Test 2/2

The *acid* test (“[...] STL algorithms [...] exist to implement in-place stable sort” – Sean Parent, No Raw Loops (2023))

```
multi::array<int, 2> A = {{ 8, 6, 7},  
                          {30, 1, 2},  
                          { 8, 9, 11}};  
  
std::sort( A.begin(), A.end() );  
std::is_sorted( A.begin(), A.end() );  
// {{ 8, 6, 7},  
// { 8, 9, 11},  
// {30, 1, 2}}
```

godbolt.org/z/eGK1T5j4M

Passes!

Indicates that our iterators work correctly with `swap`, `std::rotate`, `stable_partition`

The litmus/acid tests are similar but prove different things

Looks complicated, does it work? Bonus Test

```
multi::array<int, 2> A = {{ 8, 6, 7},  
                          {30, 1, 2},  
                          { 8, 9, 11}};  
std::sort( A.transposed().begin(), A.transposed().end() );  
assert( std::is_sorted( A.transposed().begin(), A.transposed().end() ) );  
// {{6, 7, 8},  
//   {1, 2, 30},  
//   {9, 11, 8}}
```

godbolt.org/z/1zdEfnPK9

Passes!

Indicates that our iterators work correctly with `swap`, `std::rotate`, `stable_partition`

Also works with `std::ranges` algorithms, filters, etc.

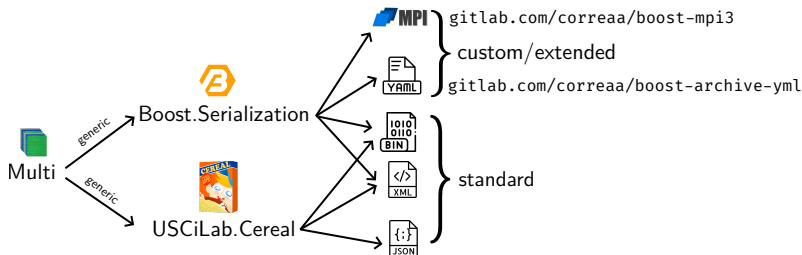
Generic Serialization (Boost.Serialization and Cereal)

```
#include <boost/archive/xml_oarchive.hpp>
```

```
multi::array<int, 2> A({10, 10});  
std::ofstream ofs{"data.xml"};  
boost::archive::xml_oarchive xoa{ofs};  
xoa << boost::make_nvp("A", A);
```

```
#include <boost/archive/xml_iarchive.hpp>
```

```
multi::array<int, 2> B;  
std::ifstream ifs{"data.xml"};  
boost::archive::xml_iarchive xia{ifs};  
xia >> B;
```



`multi::array<T, D, Allocator>` — the allocator controls where memory lives:

Standard:

- Classic (stateless allocators)
- Scoped allocators (propagated nested allocators)
- Polymorphic allocators `std::pmr::polymorphic_allocator` → `multi::pmr::array<T,D>`

Fancy:

- Interprocess Allocator (memory mapped) `boost::interprocess::allocator`

Special purpose:

- `thrust::device_allocator` → `multi::thrust::device_array<T,D>` (device memory)
- `thrust::universal_allocator` → `multi::thrust::universal_array<T,D>` (unified memory)
- `thrust::mr::allocator` (with custom memory resources)

`std::allocator_traits<A>::pointer` is not always `T*`:

Allocator	Pointer type	Memory
<code>std::allocator<T></code>	<code>T*</code>	CPU
<code>std::pmr::polymorphic_allocator<T></code>	<code>T*</code>	PMR
<code>thrust::device_allocator<T></code>	<code>thrust::device_ptr<T></code>	GPU
<code>thrust::universal_allocator<T></code>	<code>thrust::universal_ptr<T></code>	CPU+GPU
<code>thrust::omp::allocator<T></code>	<code>thrust::omp::pointer<T></code>	OpenMP
<code>boost::interprocess::allocator<T,...></code>	<code>boost::interprocess::offset_ptr<T></code>	Shared mem

[
label=allocator3] From WHERE to HOW The library propagates the pointer type throughout subarrays, iterators, and references; all carry the allocator's pointer. There is compile-time information on HOW to run.

Both the library and the user can exploit this information.

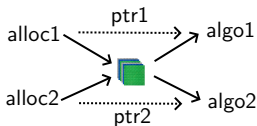
```
multi::array<int, 2> A({10, 10});  
multi::array<int, 2> B({10, 10});
```

```
B = A;
```

```
using alloc = thrust::device_allocator<int>;  
multi::array<int, 2, alloc> A({10, 10});  
multi::array<int, 2, alloc> B({10, 10});
```

```
B = A;
```

```
template<typename T, dimensionality_type D, class Alloc>  
class array {  
    using element_ptr = std::allocator_traits<Alloc>::pointer;  
    auto operator=(array const& other) → array& {  
        adl_copy_n(other.elements().begin(), other.elements().end(), elements().begin());  
    }  
};
```



Algorithm Dispatching inside the library

```
class adl_copy_n_t {  
    auto _(priority<0>, auto... as) ... {return std::copy_n(...);}    // CPU fallback  
    auto _(priority<1>, auto... as) ... {return thrust::copy_n(...);} // GPU  
    auto _(priority<2>, auto... as) ... {return copy_n(...);}      // ADL  
    auto _(priority<3>, auto it, auto... rst) ... {return it.copy_n(...);} // member  
public:  
    template<class... As> auto operator()(As&&... as) const → decltype(...) {  
        return _(priority<3>(), std::forward<As>(as)...);  
    }  
};  
inline constexpr adl_copy_n_t adl_copy_n;
```

- Overload resolution picks the highest priority that compiles (SFINAE)
- Same call site works for CPU (`std::`), GPU (`thrust::`), or user types
- Extensible (intrusive): add members: add a `copy_n` member function

Lazy sequences, the evolution of iota

Standard:

```
std::iota(c.begin(), c.end(), 0); // c = {0, 1, 2, 3, ...}
```

```
std::views::iota(1, 10); // {1, ..., 9}
```

In the library:

```
multi::array<char, 1> A = ...;  
A // {'a', 'b', 'c', 'd'}  
A.extent(); // { 0, 1, 2, 3}  
A.size(); // 4
```

```
multi::array<int, 2> B = ...;  
B.extent(); // leading dimension {0, 1, 2, 3, ..., B.size() - 1}
```


Lazy array, multidimensional iota

```
multi::array<char, 2> B = ...;  
B.extents(); // leading dimension {0, 1, 2, 3, ..., B.size() - 1}
```

```
multi::array<char, 2> B = {  
    {'a', 'b', 'c'},  
    {'d', 'e', 'f'}  
};  
B.extents(); // structured Cartesian product  
% {  
%   { tuple{0, 0}, tuple{0, 1}, tuple{0, 2} }, % <-- B.extents().begin()  
%   { tuple{1, 0}, tuple{1, 1}, tuple{1, 2} }  
% }  
  
B.extents().elements(); % {tuple{0, 0}, tuple{0, 1}, tuple{0, 2}, tuple{1, 0},...}  
B.extents().elements().begin() -- ... -- .end()  
  
% std::views::cartesian_product( get<0>(B.extents()), get<1>(B.extents()) );
```

Initialize from coordinates function

Re: partially formed elements in containers



○ Sean Parent <sparent@adobe.com>

Wednesday, May 17, 2023 at 2:17 PM

To: Correa Tedesco, Alfredo ^

This is fine - though you might make uninitialized an option. You might also look at what would be required to do initialization on construction - for example, being able to provide generators or transform functions from coordinate to value.

🤔 `multi::array<int, 2> A({10, 10}); set_from_coordinates(A, [](auto i, auto j){
return i + j; })); ???`

🤔 `multi::array<int, 2> const A({10, 10}, [](auto i, auto j){ return i + j; })); ???`

💡 `multi::array<int, 2> const A( ...special range with coordinates... ); A = ...`

Restriction (mathematics)

from Wikipedia

[...] In mathematics, the restriction of a function f is a new function, denoted $f|_A$ or $f \restriction A$, obtained by choosing a smaller domain A for the original function f . The function f is then said to extend $f \restriction A$. [...]

Translated to C++, f could be a function object that maps coordinates to values, and A can be a Cartesian set of points (of the array)

```
multi::???({2, 3}, [](auto i, auto j) { return i + j; });

[](auto i, auto j) { return i + j; } ^ multi::extents_t{2, 3};

% {
%   { f(0, 0), f(0, 1), f(0, 2) }, % <-- .begin()
%   { f(1, 0), f(1, 1), f(1, 2) }
% }
```

General lazy arrays: function restricted to a domain

A `multi::restriction` is the restriction (in the mathematical sense) of a function to a bounded index domain.

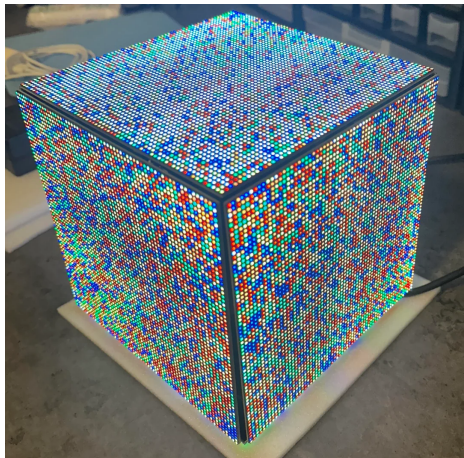
```
% lazy 2x3 array R[i][j] = i + j, no memory allocated
auto R = multi::restriction({2, 3}, [](auto i, auto j) { return i + j; });

% equivalent idiomatic form using the ^ operator ("restricted to")
auto R = [](auto i, auto j) { return i + j; } ^ multi::extensions_t{2, 3};

R[1][2];           % 3           (computed on access)
R.size();           % 2
R.num_elements();  % 6
+R;                % materialize into multi::array<int, 2>
```

Models the Multi array concept — it has extents, iterators, `operator[]`, etc. Structural operations compose (still) without allocating: transposed, repeated, diagonal, partitioned, reversed, rotated, transformed, `element_transformed`.

Take a $64 \times 64 \times 64$ array ($\sim 250k$ elements) A , apply a compute-bound op.



Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

Index-based (coordinate access)

```
auto [is, js, ks] = A.extents();  
for(auto i : is)  
  for(auto j : js)  
    for(auto k : ks)  
      A[i][j][k] = op(A[i][j][k]);
```

Range-for (element access)

```
for(auto&& plane : A)  
  for(auto&& row : plane)  
    for(auto&& elem : row)  
      elem = op(elem);
```

timing: **2.6 msec**

Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

```
std::for_each(      A.begin(), A.end(),
    [](auto&& plane) {
        for(auto&& row : plane) {
            for(auto&& elem : row) {
                elem = op(elem);
            }
        }
    }
);
```

timing: **2.6 msec**

Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

```
std::for_each(std::execution::par, A.begin(), A.end(),
    [](auto&& plane) {
        for(auto&& row : plane) {
            for(auto&& elem : row) {
                elem = op(elem);
            }
        }
    }
);
```

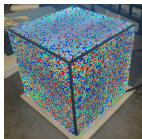
timing: **1.4 msec**

Algorithms (attempt to parallelize in the GPU)

Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

```
thrust::for_each(thrust::cuda::par, A.begin(), A.end(),
    [](auto&& plane) __device__ {
        for(auto&& row : plane) {
            for(auto&& elem : row) {
                elem = op(elem);
            }
        }
    }
);
```

timing: doesn't compile! (Thrust is unfriendly to proxy objects)



But even if it did compile...

Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

```
thrust::for_each(thrust::cuda::par, A.elements().begin(), A.elements().end(), []  
    __device__(auto& e) {  
    e = op(e);  
});
```

timing: **0.3 msec**

Take a 64x64x64 array ($\sim 250k$ elements) A, apply a compute-bound op.

```
std::for_each(std::execution::par, A.elements().begin(), A.elements().end(), [] (
    auto& e) {
    e = op(e);
});
```

timing: **0.7 msec**

Algorithm	Time (ms)	Speedup
Nested loops	2.6	1.0x
STL (leading dimension)	2.6	1.0x
Parallel STL (leading dimension)	1.4	1.9x
thrust::for_each ([cuda::par,] flat elements)	0.3	8.7x
std::for_each (std::par, flat elements)	0.7	3.7x
std::for_each (flat elements)	3.0	0.9x

Prefer algorithms to raw loops. Prefer to fuse loops.

```
algorithm(... A.elements() ...);
```

Task: assign values based on the 3D indices

```
auto [is, js, ks] = A.extents();  
for(auto i : is) for(auto j : js) for(auto k : ks)  
    A[i][j][k] = fun(i + j + k);
```

```
std::transform(  
    A.extents().elements().begin(), A.extents().elements().end(),  
    A.elements().begin(),  
    [](auto ijk) { auto [i, j, k] = ijk;  
        return fun(i + j + k);  
    }  
);
```

Note: `A.extents().elements().size() == A.elements().size()`

Recipe to 'port' your array code

Convert your loops into algorithms (discover)



Flatten your algorithms (`.elements()`, `.extensions()`, lazy arrays)



Parallelize your algorithm (`std::execution::par`)



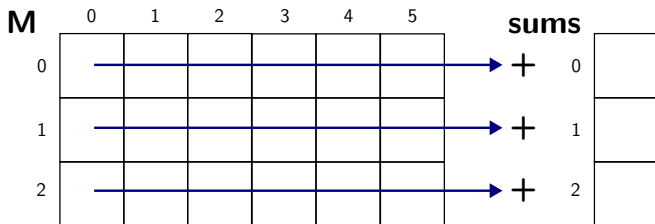
Change your memory space (use custom libraries if necessary, e.g. Thrust)

The Problem: Row-wise Reduction

Given a 2D array, reduce each row:

```
auto M      = multi::array<double, 2, thrust::device_allocator<double> >({100, 1000});
auto sums   = multi::array<double, 1, thrust::device_allocator<double> >(M.size(0));

thrust::transform(M.begin(), M.end(), sums.begin(),
    [](auto const& row) {
        return thrust::reduce(row.begin(), row.end());
    }
);
```



Solution: use Thrust's `reduce_by_key`

🏠 > CUDA C++ Core Libraries > Thrust > Thrust API documentation > Algorithms > Reductions
> `thrust::reduce_by_key`

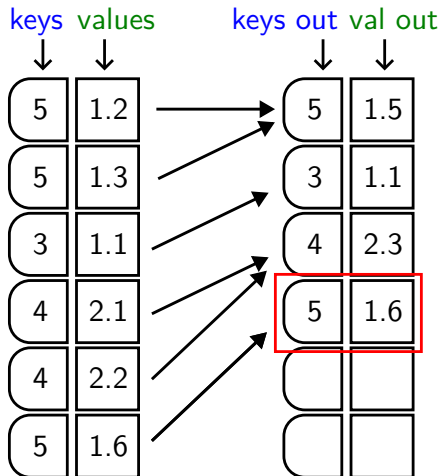
`thrust::reduce_by_key`

Overloads

```
reduce_by_key(exec, keys_first, keys_last, values_first,  
keys_output, values_output)
```

```
template<typename DerivedPolicy, typename InputIterator1, typename  
InputIterator2, typename OutputIterator1, typename OutputIterator2>  
::cuda::std::pair<OutputIterator1, OutputIterator2> thrust::reduce_by_key(  
    const thrust::detail::execution_policy_base<DerivedPolicy> &exec,  
    InputIterator1 keys_first,  
    InputIterator1 keys_last,  
    InputIterator2 values_first,  
    OutputIterator1 keys_output,  
    OutputIterator2 values_output  
)
```

`reduce_by_key` is a generalization of `reduce` to key-value pairs. For each group of consecutive keys in the range `[keys_first, keys_last)` that are equal, `reduce_by_key` copies the last element of the group to the `keys_output`. The corresponding values in the range are reduced using the `plus` and the result copied to `values_output`.



Solution: Implement reduce_by_row

```
template<class ExecutionPolicy, class T, class S>
auto reduce_by_row(ExecutionPolicy&& ep, T const& M, S&& sums) → S&& {
    assert(M.extension() == sums.extension());
    auto row_idx = []__host__ __device__ (int i, int j){return i;} ^ M.extensions();

    ::thrust::reduce_by_key(
        ep,
        row_idx.elements().begin(), row_idx.elements().end(),
        M.elements().begin(),
        ::thrust::make_discard_iterator(),
        sums.begin()
    );

    return std::forward<S>(sums);
}
```

All $N_x \times N_y$ elements processed in parallel; no serial loop over rows.

reduce_by_index: lazy arrays

Works directly on **restrictions** — no intermediate storage:

```
multi::array<double, 1, thrust::device_allocator<double> > sums(nx);

reduce_by_row(
    [](multi::index ix, multi::index iy) { return ix * iy; }
    ^ multi::extensions_t<2>(nx, ny),    % lazy 2D array
    sums
);
```

- No `multi::array<double, 2>` ever allocated
- Values generated on-the-fly by each GPU thread
- Combines lazy evaluation with massively parallel reduction

Recipe to 'port' your array code (revisited)

Convert your loops into algorithms (discover)



Flatten your algorithms (`.elements()`, `.extensions()`, lazy arrays)



Parallelize your algorithm (`std::execution::par`)



Change your memory space (use custom libraries if necessary, e.g. Thrust)



Keep looking for algorithms

Boost.Multi ships optional adaptors that integrate arrays with external libraries:

Header	Library	Purpose
<code>adaptors/blas.hpp</code>	BLAS (cuBLAS, rocBLAS)	matrix/vector algebra
<code>adaptors/lapack.hpp</code>	LAPACK	linear algebra (SVD, etc.)
<code>adaptors/fftw.hpp</code>	FFTW3	FFT on CPU
<code>adaptors/cufft.hpp</code>	cuFFT	FFT on NVIDIA GPU
<code>adaptors/thrust.hpp</code>	Thrust	GPU/OMP parallel algorithms
<code>adaptors/tblis.hpp</code>	TBLIS	tensor contractions
<code>adaptors/mpi.hpp</code>	MPI	distributed arrays

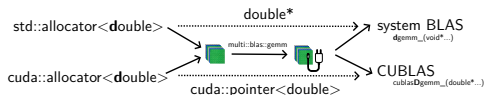
Adaptors are header-only and opt-in. Include only what you use.

- When possible, the same code should work for different backends
- Uses functional style of programming (deviates from `std::linalg`)
- Efficient (no surprises)

```
multi::array<double, 2>  
  A({3, 3}), B({3, 3}), C({3, 3});
```

```
multi::array<double, 2, thrust::  
  device_allocator<int>>  
  A({3, 3}), B({3, 3}), C({3, 3});
```

```
C = blas::gemm(1.0, A, B); // C = 1.0*A*B, shouldn't allocate  
A = + blas::gemm(1.0, A, B); // operator+ to break aliasing
```



`multi::blas::gemm` produces a "gemm" ranges; custom `copy_n` calls actual GEMM operation

```
// external_libs/gpurun/include/gpu/array.hpp:112

template<class type, size_t dim,
#ifdef ENABLE_GPU
    class allocator = caching_allocator<type>    // unified GPU memory
#else
    class allocator = std::allocator<type>      // ordinary CPU heap
#endif
>
using array = boost::multi::array<type, dim, allocator>;
```

- `gpu::array<T,N>` is a `boost::multi::array<T,N,Alloc>`
- The allocator switches at compile time: GPU builds use `thrust::universal_allocator` (visible to both CPU and GPU); CPU builds use `std::allocator`
- All multi indexing, slicing, and algorithm support is inherited unchanged
- No code change needed between CPU and GPU paths

```
// Arrays of physical quantities -- element type can be anything
multi::array<vector3<double>, 1> forces(geo.size());
multi::array<double, 1>          charges(geo.size());

for(int ii = 0; ii < geo.size(); ii++)
    charges[ii] = atomic_pot
        .pseudo_for_element(geo.species(ii))
        .valence_charge();

interaction_energy(geo.size(), cell, charges,
                  geo.positions(), atomic_pot.range_separation(),
                  energy, forces);
```

Multi arrays are generic: `vector3<double>` is a valid element type, giving a 1D array of 3-vectors representing per-atom forces.

src/ionic/interaction.hpp:317

```
gpu::array<double, 2> eigenvalues_  
gpu::array<double, 2> occupations_  
...  
// 2D arrays sized at runtime from MPI-distributed k-point data  
eigenvalues_.reextent({n_kpin, max_local_spinor_set_size_});  
occupations_.reextent({n_kpin, max_local_spinor_set_size_});
```

- reextent resizes without invalidating the element type
- Dimensions come from the MPI partition of k-points and states
- The same syntax works for CPU and GPU arrays (gpu::array)

src/systems/electrons.hpp:188


```
// phi is a 4D array: [ix][iy][iz][istate]  
// vector field stores complex3 elements; cast to scalar complex  
// then rotate axes so [istate] becomes the leading dimension  
  
auto&& fphi_as_scalar =  
    fphi.hypercubic()  
        .template reinterpret_array_cast<complex>(3)  
        .rotated().rotated().rotated() // bring [istate] to front  
        .flatted() // merge spatial dims  
        .rotated(); // final layout for FFT  
  
to_fourier_array(phi.basis(), fphi.basis(),  
                 phi_as_scalar, fphi_as_scalar);
```

No data is copied: every operation returns a zero-cost view with a different layout. The FFT routine sees a plain 2D array.

```
// operations/transform.hpp:23-25, 68-104

// {true,true,true,false}: transform dims 0,1,2 -- batch over dim 3 (states)
fft::dft_forward({true, true, true, false}, array_rs, array_fs);
fft::dft_backward({true, true, true, false}, array_fs, array_rs);

// MPI-parallel: split into 1D + 2D passes with a transpose in between
fft::dft_forward({true, false, false, false}, array_rs, tmp);
parallel::transpose_forward(comm, ...);
fft::dft_forward({true, true, false, false}, tmp, array_fs);

fft::dft_backward({true, true, false, false}, array_fs, tmp);
parallel::transpose_backward(comm, ...);
fft::dft_backward({true, false, false, false}, tmp, array_rs);
```

- The boolean mask selects *which dimensions* to transform
- `fft::` and `fftw::` share the same interface — only the `#include` and namespace alias change
- MPI-distributed FFT is decomposed manually into 1D passes separated by an all-to-all transpose

Operation	Out-parameter form	Functional form
<i>Level 1 — scalars from vectors</i>		
nrm2 $\ x\ _2$	blas::nrm2(x, r)	double r = blas::nrm2(x)
asum $\ x\ _1$	blas::asum(x, r)	double r = blas::asum(x)
dot $x \cdot y$	blas::dot(x, y, r)	double r = blas::dot(x, y)
<i>Level 1 — vector updates</i>		
axpy $y += \alpha x$	blas::axpy(a, x, y)	$y += \text{blas::axpy}(a, x)$
<i>Level 2 — matrix-vector</i>		
gemv $y = \alpha Ax + \beta y$	blas::gemv(a, A, x, b, y)	$y = \text{blas::gemv}(a, A, x)$
<i>Level 3 — matrix-matrix</i>		
gemm $C = \alpha AB$	blas::gemm(a, A, B, b, C)	$C = \text{blas::gemm}(a, A, B)$
herk $C = \alpha AA^\dagger$	blas::herk(fill, a, A, b, C)	$C = \text{blas::herk}(a, A)$

View manipulators (usable as arguments above): blas::H(A) — conjugate-transpose blas::T(A) — transpose blas::C(x) — conjugate

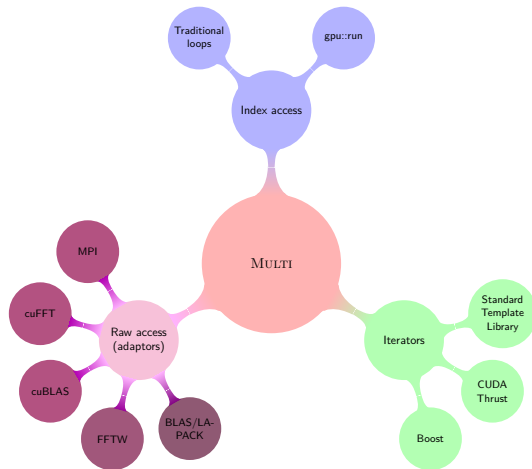
```
// operations/overlap.hpp:33-45
namespace blas = boost::multi::blas;

// Overlap matrix:  $S_{ij} = dV * \sum_k \text{conj}(\text{phi1}[k][i]) * \text{phi2}[k][j]$ 
//   blas::H() -- conjugate transpose (Hermitian adjoint)
//   gemm()    -- dispatches to BLAS (CPU) or cuBLAS (GPU)

// serial case: single gemm into the result block
olap.block() = blas::gemm(basis.volume_element(),
                          blas::H(phi1_matrix),
                          phi2_matrix);

// parallel case: compute locally, then MPI-reduce by block
auto arr = +blas::gemm(basis.volume_element(),
                      blas::H(phi1_matrix),
                      phi2_matrix);
```

- `blas::gemm` and `blas::H` work on any `multi` array, CPU or GPU — no code change needed
- The `+` prefix forces immediate evaluation into an owning array
- The same adaptor also provides `blas::nrm2`, `blas::gemv`, `blas::trsm`, ...



Thoughts about AI

- No AI was used to develop the library
- After Boost Review (conditionally accepted), AI was used to proofread documentation
- The irony isn't lost on me: I'm probably documenting mainly for AI
- ... but I'd prefer AI that learns to code using Generic Programming

- 1 **Generic programming works for arrays.** (If you make some concessions 🌶️)
The same code handles 1D...ND, CPU and GPU, owned and borrowed memory.
- 2 **Views are cheap, copies are explicit.** (no need for reference counting or pointer/shallow semantics)
Slicing, transposing, striding — all return lightweight views over the same data.
- 3 **Compose/expand with the STL, don't replace it with frameworks**
Standard algorithms, execution policies, and iterators work out of the box.
- 4 **Allocator = backend**
Swap the allocator to move between CPU, CUDA, OpenMP — no code changes.

Documentation

`correaa.gitlab.io/boost-multi`

`correaa.github.io/boost-multi`